PRD: API Change Guardian

Author: Kavish
Status: Draft v1
Target role: Agentic AI Engineer Intern (portfolio project)
Scope level: MVP+ (about 7/10 technical depth)
Timeline: about 3 weeks

1. Overview

API Change Guardian is a GitHub App that watches the changelogs and release notes of popular third-party APIs and SDKs (OpenAI, Stripe, Twilio), finds the code in a repository that is actually affected by each change, and opens a draft pull request with a verified fix. Developers hear only about changes that affect their code, and they receive a ready-to-review diff instead of an alert to research.

2. Problem Statement

Modern software depends on external providers that regularly rename fields, deprecate endpoints, change response formats, and release new features. Missed changes cause production breakage, often at inconvenient times. The pitch behind this idea cites that over 30% of downtime at AWS traced back to unnoticed external API and package changes. This figure comes from the pitch and has not been independently verified, so it should be described as reported in any public write-up.

Changelogs exist, but no team has time to read dozens of them for every dependency. Existing tools such as Dependbot and Renovate bump versions but do not understand whether a breaking change touches your code or rewrite the affected call sites.

3. Goals and Non-Goals

Goals

- Detect new breaking or deprecating changes from 3 providers within 24 hours of release.
- Report only changes that affect the specific repository (low noise).
- Generate a minimal, verified fix as a draft PR.

- Measure every stage with explicit evals and publish the results.
- Demonstrate safe agent design: human approval, sandboxing, and untrusted-input handling.

Non-Goals (for this version)

- Supporting more than one programming language (Python only at first).
- Monitoring more than about 3 to 5 providers.
- Auto-merging any PR.
- Type-aware scanning of dynamically constructed or deeply wrapped clients.
- Handling behavioral changes that cannot be expressed as a code edit.
- A multi-tenant SaaS with billing and a dashboard.

4. Target Users

Persona	Need
Solo developer or small team using several SaaS APIs	Avoid surprise breakage without reading changelogs
Maintainer of an open-source project	Stay current with SDK migrations cheaply
Recruiter or hiring engineer (secondary audience)	Quickly see engineering depth, evals, and safe design

5. User Stories

1. As a developer, I install the GitHub App on my repo so it can monitor my dependencies.
2. As a developer, I receive a draft PR only when a provider change affects code I actually wrote.
3. As a reviewer, I see the changelog link, affected files, a confidence level, and test results in the PR description.
4. As a developer, if the agent cannot produce a verified fix, I get an issue explaining the impact instead of a broken PR.
5. As a developer, I can close a PR and the app does not reopen it for the same change.
6. As a maintainer, I can see which changes were detected, skipped, and why.

6. Functional Requirements

FR1: Change ingestion

- Poll GitHub Releases and CHANGELOG files for the openai-python, stripe-python, and twilio-python SDKs on a daily schedule.
- Use ETags and conditional requests to respect rate limits.
- Deduplicate entries by a hash of provider, version, and content.
- Store raw release text for auditing.

FR2: Structured extraction

- Use an LLM with a strict JSON schema to convert release notes into change records (provider, package, affected versions, change type, qualified symbol, replacement, breaking flag, sunset date, evidence quote, confidence).
- Validate with a schema library and retry on failure.
- Reject any record whose evidence quote does not appear verbatim in the source text.
- Treat release text strictly as data, never as instructions to the model.

FR3: SDK API-surface diffing (ground truth)

- For SDK-level changes, compare the public API surface of consecutive package versions (for example with griffe) to confirm removed or renamed symbols.
- Prefer the diff result when it disagrees with the LLM extraction; use the LLM for the explanation and migration hint.

FR4: Repository scanning

- Read the repo's pinned versions from requirements.txt, poetry.lock, or equivalent, and only consider changes whose affected version range includes the pinned version.
- Parse Python files with tree-sitter and build an import and alias table.
- Track simple client assignments (for example, a variable bound to a client class) and resolve attribute chains to fully qualified names.
- Match call sites to the change's symbol and check for the affected parameter.
- Emit findings with file, line, matched symbol, and confidence.
- If nothing matches, produce no alert.

FR5: Fix generation

- Apply deterministic codemods (libcst) for simple cases such as parameter renames and import path changes.
- For other cases, call an LLM with the change record, migration hint, and the enclosing function, and request a unified diff limited to the call site.
- Enforce a diff size limit and restrict edits to files that contain findings.

FR6: Verification

- Run in a sandbox (Docker, no network, no secrets): syntax check, linter, type check, and the repo's test suite when available.
- Retry with error output up to 2 attempts.
- On continued failure, open an issue describing the impact instead of a PR.

FR7: GitHub integration

- Implement as a GitHub App with installation tokens, not personal access tokens.
- Request minimum permissions: Contents (read and write), Pull requests (write), Issues (write), Metadata (read).
- Verify webhook signatures on every request.
- Create a branch and open a draft PR containing: summary, changelog link, affected files, verification results, and a confidence note.
- Keep one PR per repository and change; remember closed PRs and do not reopen them.

7. Non-Functional Requirements

Area	Requirement
Safety	Never push to the default branch; PRs are always drafts; human merges
Security	Sandbox untrusted repo code; no secrets in the sandbox; treat changelog text as untrusted input
Reliability	Idempotent jobs; retries with backoff; handle secondary rate limits

Area	Requirement
Observability	Log every LLM call (inputs, outputs, tokens, latency) and cost per repository
Performance	Scan a mid-size repo (about 500 files) in under 2 minutes with caching
Cost	Extraction runs once per release note; fix generation capped per finding

8. System Architecture

```
Scheduler -> Ingest workers -> Extract workers (LLM) -> Changes DB
                                                            |
GitHub webhook -> Job queue -> Scan worker -> Fix worker (sandbox) -> Draft
PR / Issue
```

Components: scheduler, ingestion workers, LLM extractor, API-surface differ, repo scanner, fixer (codemods plus LLM), sandbox runner, GitHub App service.

Suggested stack: Python, Pydantic, tree-sitter, libcst, griffe, Postgres (or SQLite for the MVP), Redis with RQ or Celery, Docker, a GitHub App via the REST API, and an LLM API with structured output.

Core tables: changes, repos, findings, pull_requests, llm_calls.

9. Evaluation Plan

Each stage is evaluated separately so failures can be traced.

Eval	Dataset	Metrics	Target
Extraction	40 to 50 hand-labeled release notes	Precision and recall of (symbol, change type); hallucination rate	Precision at least 0.85; hallucination under 5%
Scanner	15 to 20 real repos with a known breaking change	Call-site recall and precision; compare grep vs AST vs AST plus import resolution	Recall at least 0.85; precision at least 0.90

Eval	Dataset	Metrics	Target
Fix	Same repo set	Applies cleanly; passes lint and type check; passes tests; matches human fix; false-confidence rate	At least 60% fully verified fixes
End to end	Replay of past breaking changes	Alerts per repo per month; time from release to PR	Under 2 alerts per repo per month

Publish a results table and the grep vs AST comparison chart in the README.

10. Milestones (about 3 weeks)

Week	Deliverables
1	Ingestion for 3 SDKs; extraction with schema validation and evidence check; Postgres or SQLite storage; extraction eval set started
2	Python scanner with import and alias resolution; version gating; API-surface differ; scanner eval on 15 repos
3	Codemods plus LLM fixer; sandbox verification; GitHub App and draft PR flow; fix eval; README, architecture diagram, 2 to 3 minute demo video

Stretch goals (only if time remains): type-aware scanning with Pyright, TypeScript support, a small dashboard of detected changes.

11. Success Metrics

- Working end-to-end demo on at least 2 real repositories.
- Published eval numbers for extraction, scanning, and fixing.
- README with architecture diagram, limitations section, and demo video.
- Public repository with tests and a one-command local setup.

12. Risks and Mitigations

Risk	Impact	Mitigation
LLM misreads vague release notes	Wrong or missed changes	Evidence-quote check, API-surface diff as ground truth, confidence thresholds
Scanner misses wrapped or dynamic usage	Missed alerts	Document limits; optional Pyright stretch goal
Generated fix breaks behavior	Bad PR	Sandbox verification, small-diff limit, draft PRs, human review
Prompt injection via changelog text	Unsafe model behavior	Treat text as data, narrow permissions, no tool access during extraction
Running untrusted repo tests	Security exposure	Docker sandbox, no network, no secrets
Scope creep	Unfinished project	Python only, 3 providers, fixed milestones
Hard to find labeled eval repos	Weak evaluation	Start with open-source repos affected by well-known SDK migrations

13. Open Questions

- Which 3 providers give the best mix of well-documented breaking changes (OpenAI, Stripe, Twilio, or another)?
- Should the app support dated API versions (as with Stripe) in addition to SDK versions in the MVP?
- Hosted deployment or local-only demo for the portfolio?
- Which LLM provider and model will be used for extraction and fixing, given cost limits?

14. Portfolio Presentation Checklist

- Clear README with problem, solution, and results table

- Architecture diagram
- Short demo video showing a real PR being created
- Documented limitations and safety design
- Eval methodology and raw labeled data in the repo